2026-11-01 11:18:56 You

i want for ll responsiblity u just gave me 2 give me all in one response

ChatGPT gpt-6-mini

## Complete DevOps Resume Responsibilities – Client Interview Preparation

We'll cover all **12 responsibilities in one consolidated guide**, using the same format as the first two:

1. Simple technical definition.
2. Storytelling – Project explanation.
3. Real-world task with an incident.
4. Common errors and troubleshooting.

The project scenario remains consistent: an enterprise application serving lakhs of users across India, hosted on AWS, using Linux, Ansible, Jenkins, Docker and Kubernetes/EKS.

The explanations below are designed to help you understand and discuss realistic engineering workflows. Treat the project stories and incidents as scenarios to prepare for, not claims of personal production experience unless they reflect work you actually performed.

---

## PART 1 – ANSIBLE

### Responsibility 1: Developing and managing reusable Ansible playbooks and roles

Core concept: Automation + Reusability

**Resume statement:**

Experienced in developing, structuring and managing reusable Ansible playbooks and roles, using tasks, handlers, variables, Jinja2 templates and conditional execution to standardize automation workflows.

#### 1. Simple technical definition

Ansible is an agentless automation and configuration management tool. It connects to remote Linux servers using SSH and executes predefined tasks.

- Playbook: YAML file defining automation instructions.
- Role: Reusable collection of tasks, variables, handlers and templates.

- Tasks: Individual operations such as installing packages.
- Handlers: Triggered actions, such as restarting services when configuration changes.
- Variables: Dynamic values used in automation.
- Jinja2: Template engine used to generate configuration files dynamically.
- Conditions: Execute tasks based on particular conditions.

Typical role structure:

```
</> Plain text

roles/
  nginx/
    tasks/main.yml
    handlers/main.yml
    defaults/main.yml
    templates/nginx.conf.j2
```

**2. Storytelling – Project explanation**

"In our project, we had multiple Linux servers supporting different application environments. Initially, several configurations were repeated across servers, making maintenance difficult.

We structured our automation using reusable Ansible roles. For example, we maintained separate roles for application dependencies, web server configuration, monitoring agents and user management.

We used variables for environment-specific configurations, Jinja2 templates for dynamically generating configuration files and handlers to restart services whenever configuration changes occurred.

This allowed us to reuse existing automation rather than writing separate playbooks for every server."

**3. Real-world task with an incident**

**Task: Configure Nginx across multiple application servers.**

Workflow:

1. Create an Ansible role.
2. Define Nginx port and worker variables.
3. Install Nginx.
4. Generate configuration using Jinja2.
5. Notify a handler if configuration changes.
6. Restart and validate Nginx.

Example:

```yaml
- name: Install nginx
  package:
    name: nginx
    state: present

- name: Configure nginx
  template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
  notify: Restart nginx
```

Handler:

```yaml
- name: Restart nginx
  service:
    name: nginx
    state: restarted
```

**Incident: Nginx fails after a configuration update.**

- Check Ansible output.
- Validate configuration using `nginx -t`.
- Check `systemctl status nginx`.
- Inspect error logs.
- Correct the template and rerun the playbook.

### 4. Common errors and issues

| Error | Cause | Resolution |
|---|---|---|
| Role not found | Incorrect role path | Verify role structure |
| Undefined variable | Missing variable | Check variable declaration |
| Handler not triggered | No reported change or incorrect notification | Check `notify` |
| YAML error | Incorrect indentation | Syntax check |
| Template failure | Incorrect Jinja2 syntax | Validate `.j2` |

| | | |
|---|---|---|
| Permission denied | Insufficient privileges | Use `become: yes` |
| Repeated changes | Non-idempotent automation | Use Ansible modules properly |

**Useful commands:**

```bash
ansible-playbook site.yml --syntax-check
ansible-playbook site.yml --check
ansible-playbook site.yml --list-tasks
ansible-playbook site.yml -vvv
```

**Memory connection:** Playbook → Roles → Tasks → Variables/Templates → Handlers → Target servers.

---

## Responsibility 2: Linux server automation using Ansible

Core concept: Linux administration

**Resume statement:**

Solid knowledge of Linux server automation using Ansible, including OS-level configurations, package and dependency management, user administration, filesystem permissions, systemd service management and security configurations.

### 1. Simple technical definition

Linux automation using Ansible means managing multiple Linux servers remotely through playbooks instead of performing repetitive administrative activities manually.

| Area | Meaning |
|---|---|
| OS configuration | Hostname, timezone and system settings |
| Package management | Install, update and remove software |
| Dependency management | Install required libraries and runtimes |
| User administration | Create users and groups |
| Permissions | Manage ownership and file access |
| Systemd | Start, stop and enable services |
| Security | SSH, firewall, sudo and SELinux configuration |

### 2. Storytelling – Project explanation

"Our application infrastructure consisted of multiple Linux servers across development, QA and production.

We used Ansible to automate common Linux administration activities such as installing Java and other dependencies, creating application users, configuring directory permissions and managing systemd services.

For example, whenever a new application server was introduced, we could execute a playbook to install the required packages, configure application directories and ensure the necessary services were running.

We also maintained common security configurations to keep server configurations consistent."

**3. Real-world task with an incident**

**Task: Prepare a RHEL application server for Java deployment.**

Example playbook:

```yaml
- name: Prepare application servers
  hosts: app_servers
  become: yes

  tasks:
    - name: Install Java
      dnf:
        name: java-17-openjdk
        state: present

    - name: Create app user
      user:
        name: appuser
        state: present

    - name: Create app directory
      file:
        path: /opt/myapp
        state: directory
        owner: appuser
        mode: '0755'
```

Configure a systemd service for the application so that it can start automatically and recover according to the configured restart policy.

**Incident: Application doesn't start after server reboot.**

Possible causes:

- Systemd service not enabled.
- Java missing.
- Incorrect file permissions.
- Incorrect service configuration.
- Application startup dependency unavailable.

Troubleshooting:

```bash
systemctl status myapp
systemctl is-enabled myapp
journalctl -u myapp -xe
java -version
ls -l /opt/myapp
```

### 4. Common errors and issues

| Error | Cause | Resolution |
|---|---|---|
| Package installation failed | Repository unavailable | Check DNF/APT repositories |
| Permission denied | Incorrect ownership | Check `chown` and `chmod` |
| Service failed | Invalid configuration | Check systemd logs |
| Java application failed | Wrong runtime version | Verify Java version |
| SSH failed | Network/key configuration | Verify SSH and firewall |
| Disk full | Logs or files consuming storage | Use `df -h`, `du -sh` |
| SELinux denial | Security policy blocking access | Inspect audit logs |

**Important RHEL commands:**

```bash
cat /etc/redhat-release
dnf install nginx
dnf update
systemctl status nginx
getenforce
firewall-cmd --state
```

```
df -h
free -h
journalctl -xe
```

**Memory connection:** OS → Packages → Users → Permissions → Services → Security.

---

## Responsibility 3: AWS infrastructure automation using Ansible

Core concept: AWS + Ansible

**Resume statement:**

Good working knowledge of AWS infrastructure automation using Ansible, including EC2 instance configuration, security group management, application environment setup and dynamic inventory integration.

### 1. Simple technical definition

Ansible can interact with AWS infrastructure through AWS APIs and collections.

It can automate tasks such as:

- EC2 instance provisioning and configuration.
- Security group creation or modification.
- Tagging AWS resources.
- Application setup on EC2.
- Discovering EC2 instances dynamically.

Two important concepts:

**AWS modules:** Ansible modules that communicate with AWS services, often through the `amazon.aws` collection and Python's boto3 SDK.

**Dynamic inventory:** Automatically discovers AWS EC2 instances based on filters, tags, regions or other criteria rather than maintaining a static IP list.

### 2. Storytelling – Project explanation

"In our project, AWS EC2 instances were used to host application-related workloads and supporting services.

We used Ansible to automate the configuration of these instances after provisioning. This included installing dependencies, creating application directories, configuring services and applying required server settings.

We also used AWS dynamic inventory to discover EC2 instances based on their tags and environment information. This avoided maintaining IP addresses manually whenever instances were added, replaced or changed.

It helped us maintain a consistent configuration process across AWS environments."

**3. Real-world task with an incident**

**Task: Automatically configure newly launched EC2 instances.**

Example workflow:

1. EC2 instance is provisioned through Terraform or an approved AWS provisioning workflow.
2. Instance receives tags such as `Environment=QA` and `Role=Application`.
3. Ansible dynamic inventory discovers the instance.
4. Ansible connects through SSH or the configured connection mechanism.
5. Installs dependencies.
6. Configures application directories and services.
7. Validates readiness.

Dynamic inventory command:

```bash
ansible-inventory \
  -i aws_ec2.yml --graph
```

Example inventory configuration:

```yaml
plugin: amazon.aws.aws_ec2
regions:
  - us-east-1

filters:
  instance-state-name: running

keyed_groups:
  - key: tags.Environment
    prefix: env
```

**Incident: A newly launched EC2 instance is missing from Ansible inventory.**

Possible causes:

- Incorrect AWS region.
- Missing IAM permissions.
- Incorrect resource tags.
- Instance state filter mismatch.
- Dynamic inventory plugin configuration issue.

Troubleshooting:

```bash
aws sts get-caller-identity

ansible-inventory \
  -i aws_ec2.yml --list
```

Verify AWS credentials, IAM permissions, filters and tags.

#### 4. Common errors and issues

| Issue | Cause | Resolution |
|---|---|---|
| EC2 not discovered | Incorrect filter/tag | Validate inventory |
| AccessDenied | Missing IAM permissions | Review IAM policy |
| SSH failure | Key/security group issue | Check SSH connectivity |
| AWS module failure | Missing collection | Install `amazon.aws` |
| Invalid credentials | Incorrect profile/role | Check AWS identity |
| Wrong environment targeted | Incorrect tagging | Standardize tags |
| boto3 error | Dependency missing | Install required Python SDK |

**Important distinction:** Ansible can provision EC2 instances using AWS modules, but Terraform is commonly preferred for managing larger, version-controlled infrastructure lifecycles. Ansible is particularly useful for post-provisioning configuration.

**Memory connection:** AWS provisioning → EC2 discovery → Dynamic inventory → SSH → Configuration → Validation.

---

## Responsibility 4: Ansible integration with Jenkins CI/CD pipelines

Core concept: Automation pipeline integration

**Resume statement:**

Strong understanding of Ansible integration with Jenkins CI/CD pipelines, automating server configuration, application deployments and infrastructure provisioning as part of continuous delivery workflows.

**1. Simple technical definition**

Jenkins is a CI/CD automation server, whereas Ansible is a configuration management and automation tool.

They can be integrated so Jenkins automatically triggers Ansible playbooks as part of a pipeline.

For example:

- Jenkins pulls code from Git.
- Jenkins builds the application.
- Jenkins executes Ansible playbooks.
- Ansible configures target servers.
- Application is deployed.
- Jenkins validates deployment.

**2. Storytelling – Project explanation**

"In our project, we integrated Ansible with Jenkins to automate repetitive configuration and deployment activities.

Whenever a deployment was triggered, Jenkins executed the pipeline stages responsible for retrieving source code, preparing the build and invoking the required Ansible playbook.

Ansible then connected to the target servers and performed application-related configuration and deployment tasks.

We maintained credentials securely in Jenkins and passed environment-specific parameters to the automation process.

This reduced manual deployment steps and helped maintain a consistent release workflow."

**3. Real-world task with an incident**

**Task: Automatically deploy a Java application to EC2 servers.**

Workflow:

```
</> Plain text
```

```
Git → Jenkins → Build → Artifact
                    ↓
              Ansible Playbook
                    ↓
             Target EC2 Servers
                    ↓
             Service Validation
```

Example Jenkins pipeline:

```groovy
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                git 'https://github.com/org/app.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }

        stage('Deploy') {
            steps {
                sh '''
                ansible-playbook deploy.yml \
                  -i inventory/qa \
                  --private-key "$SSH_KEY"
                '''
            }
        }

        stage('Validate') {
            steps {
                sh 'ansible app_servers -m ping'
            }
        }
    }
}
```

This is illustrative; in a real Jenkins pipeline, SSH credentials should be injected securely using Jenkins credentials bindings, not exposed through plain environment variables or logs.

**Incident: Jenkins build succeeds, but Ansible deployment fails.**

Possible reasons:

- Jenkins agent cannot connect to target servers.
- Incorrect SSH credentials.
- Ansible inventory contains stale IPs.
- Target server has insufficient permissions.
- Required application directory doesn't exist.

Troubleshooting:

```bash
ansible all -i inventory/qa -m ping

ansible-playbook deploy.yml \
  -i inventory/qa -vvv
```

Check Jenkins console output, agent connectivity, SSH credentials and target-server logs.

### 4. Common errors and issues

| Issue | Cause | Resolution |
|---|---|---|
| Jenkins cannot execute Ansible | Ansible missing on agent | Install Ansible |
| SSH authentication failure | Invalid credentials | Verify Jenkins credentials |
| Host unreachable | Network/security group issue | Validate connectivity |
| Playbook failed | Incorrect YAML or variables | Syntax check |
| Build succeeds but deployment fails | Artifact path incorrect | Verify artifact |
| Wrong environment deployment | Incorrect inventory | Use controlled parameters |
| Credentials exposed | Poor secret handling | Use Jenkins credentials |

**Memory connection:** Git → Jenkins → Build → Ansible → Server → Validate.

---

## PART 2 – KUBERNETES

### Responsibility 5: Kubernetes container orchestration and cluster architecture

**Resume statement:**

Strong knowledge of Kubernetes container orchestration and cluster architecture, managing containerized application workloads, deployment lifecycle, workload distribution and application availability across distributed environments.

### 1. Simple technical definition

Kubernetes is a container orchestration platform used to deploy, manage, scale and maintain containerized applications across multiple machines.

It manages:

- Container deployment and lifecycle.
- Workload scheduling.
- Application scaling.
- Service discovery and networking.
- Self-healing and application availability.
- Configuration and secrets.
- Rolling updates and rollbacks.

A Kubernetes cluster has two major components:

| Component | Responsibility |
|---|---|
| Control Plane | Manages and controls the cluster |
| Worker Nodes | Run application workloads |

Important control plane components:

- API Server – Entry point for Kubernetes requests.
- Scheduler – Selects appropriate worker nodes for pods.
- Controller Manager – Maintains the desired cluster state.
- etcd – Stores cluster configuration and state.

Important worker node components:

- Kubelet – Communicates with the control plane and manages pods.
- Container Runtime – Runs containers.
- Kube-proxy – Implements Service networking rules where applicable.

### 2. Storytelling – Project explanation

"In our project, we used Kubernetes to manage containerized application workloads across multiple worker nodes.

The application consisted of frontend and backend components, which were deployed as separate workloads.

We used Deployments to manage application replicas, Services to provide stable communication between components and ConfigMaps and Secrets to manage application configurations.

The Kubernetes scheduler distributed pods across available worker nodes based on resource availability and scheduling requirements.

We also monitored pod health, node status and application availability to ensure that workloads remained operational."

**3. Real-world task with an incident**

**Task: Deploy and maintain a highly available application on Kubernetes.**

Imagine the application has:

- 3 frontend replicas.
- 4 backend replicas.
- Multiple worker nodes.
- Internal ClusterIP services.
- External ingress routing.

Example:

```yaml
</> YAML

apiVersion: apps/v1
kind: Deployment

metadata:
  name: backend

spec:
  replicas: 4

  selector:
    matchLabels:
      app: backend

  template:
    metadata:
      labels:
        app: backend

    spec:
```

```
      containers:
        - name: backend
          image: myapp/backend:v1
          ports:
            - containerPort: 8080
```

Kubernetes maintains the desired replica count.

**Incident: One worker node suddenly becomes NotReady.**

Imagine a worker node experiences an infrastructure issue.

What happens?

- The control plane detects node health problems.
- The node may eventually be marked NotReady.
- Pods on that node may be recreated or rescheduled depending on workload type, node failure detection, eviction timing and available capacity.
- Services route traffic to healthy available endpoints.

Troubleshooting:

```bash
kubectl get nodes -o wide

kubectl describe node worker-1

kubectl get pods -A -o wide

kubectl get events -A \
  --sort-by=.metadata.creationTimestamp
```

Check kubelet, container runtime, node resources and connectivity.

### 4. Common errors and issues

| Issue | Possible cause | Troubleshooting |
|---|---|---|
| Node NotReady | Kubelet/runtime issue | Check node services |
| Pod Pending | Scheduling constraints | Describe pod |
| CrashLoopBackOff | Application startup failure | Inspect logs |
| ImagePullBackOff | Registry/image issue | Check image and credentials |
| Node resource pressure | CPU/memory/disk exhaustion | Check node metrics |

| | | |
|---|---|---|
| API Server unavailable | Control plane problem | Check cluster health |
| Pods not distributed | Scheduling constraints | Review affinity and capacity |

Useful commands:

```bash
kubectl cluster-info
kubectl get nodes
kubectl get pods -A
kubectl get deployments -A
kubectl describe pod <pod>
kubectl logs <pod>
kubectl top nodes
```

**Memory connection:** Control Plane → Scheduler → Worker Node → Pod → Container → Service → User traffic.

---

## Responsibility 6: Managing Kubernetes workloads and deployment configurations

Core concept: Kubernetes workload management

**Resume statement:**

Experienced in managing Kubernetes workloads and deployment configurations using Deployments, StatefulSets, ReplicaSets, ConfigMaps, Secrets and Helm charts, with environment-specific configuration management.

### 1. Simple technical definition

Kubernetes workload resources manage the lifecycle of application pods.

Understand these components:

| Resource | Purpose | Real-world example |
|---|---|---|
| Deployment | Manages stateless applications | Frontend/API |
| ReplicaSet | Maintains desired pod count | Four backend replicas |
| StatefulSet | Manages stateful workloads with stable identities | Kafka, database workloads |
| ConfigMap | Stores non-sensitive configuration | Application URLs |
| Secret | Stores sensitive configuration | Passwords, tokens |
| Helm | Packages Kubernetes manifests | Environment-specific releases |

**Important:** A Deployment manages ReplicaSets, and ReplicaSets manage Pods. You generally manage the Deployment rather than creating ReplicaSets manually.

**2. Storytelling – Project explanation**

"Our application was divided into multiple components, and we used different Kubernetes workload resources based on their requirements.

For stateless frontend and backend services, we used Deployments to manage replicas and application updates.

For workloads requiring stable identities and persistent storage, StatefulSets were appropriate.

We used ConfigMaps to maintain environment-specific application configurations and Secrets for sensitive values such as credentials.

We also maintained Helm charts to standardize deployment configurations across development, QA and production environments."

**3. Real-world task with an incident**

**Task: Deploy a backend application using Deployment, ConfigMap and Secret.**

Example ConfigMap:

```yaml
</> YAML

apiVersion: v1
kind: ConfigMap
metadata:
  name: backend-config
data:
  APP_ENV: production
  API_PORT: "8080"
```

Example Secret:

```yaml
</> YAML

apiVersion: v1
kind: Secret
metadata:
  name: backend-secret
type: Opaque
stringData:
```

```
  DB_USERNAME: appuser
  DB_PASSWORD: example-password
```

This is an illustrative manifest. In production, use appropriately controlled secret management, such as AWS Secrets Manager with an integration mechanism, and avoid committing credentials to Git.

Deployment:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: backend:v1
          envFrom:
            - configMapRef:
                name: backend-config
            - secretRef:
                name: backend-secret
```

**Incident: Application deployment succeeds, but the backend cannot connect to the database.**

Possible reasons:

- Incorrect database hostname.
- Incorrect credentials.
- Secret not mounted or referenced correctly.
- Incorrect network/security group configuration.
- Database connectivity failure.

Troubleshooting:

```bash
kubectl describe pod <pod-name>

kubectl logs <pod-name>

kubectl get configmap

kubectl get secrets

kubectl get events
```

Check the environment variable references and database connectivity without exposing secret values in logs.

### 4. Common errors and issues

| Issue | Cause | Troubleshooting |
| --- | --- | --- |
| CrashLoopBackOff | Incorrect application configuration | Inspect logs |
| ConfigMap not reflected | Environment variables require pod restart | Restart workload |
| Secret unavailable | Missing or incorrect Secret | Verify reference |
| StatefulSet pod stuck | PVC or storage issue | Check PVC |
| Deployment unavailable | Pods not ready | Inspect probes |
| ReplicaSet not scaling | Insufficient capacity | Check scheduling |
| Helm release failed | Incorrect chart configuration | Run `helm lint` |

Useful commands:

```bash
kubectl get deploy,rs,pods
kubectl get statefulsets
kubectl get configmaps
kubectl get secrets
kubectl get pvc
helm list -A
```

**Memory connection:** Deployment → ReplicaSet → Pods; ConfigMap/Secret → Configuration; StatefulSet → Stable identity and storage; Helm → Package and deploy.

---

## Responsibility 7: Kubernetes networking and service discovery

Core concept: Networking + Connectivity

**Resume statement:**

Solid understanding of Kubernetes networking and service discovery, managing ClusterIP, NodePort, Load Balancer, Ingress, DNS resolution and external traffic routing for application accessibility.

### 1. Simple technical definition

Kubernetes networking allows containers, pods, services and external users to communicate with each other.

Understand the different networking components:

| Component | Purpose |
|---|---|
| ClusterIP | Internal communication within cluster |
| NodePort | Exposes service through a port on each node |
| LoadBalancer | Exposes service through an external load balancer when supported |
| Ingress | Defines HTTP/HTTPS routing rules |
| Ingress Controller | Implements those routing rules |
| CoreDNS | Resolves service DNS names |

Example:

```
</> Plain text

Frontend Pod
    |
    | backend-service
    |
CoreDNS resolves service name
    |
Backend Service
    |
Backend Pods
```

A Kubernetes Service gives stable access to changing pod IP addresses.

### 2. Storytelling – Project explanation

"In our project, frontend and backend services were deployed as separate Kubernetes workloads.

We used ClusterIP Services for internal communication between application components, so they didn't need to depend on individual pod IP addresses.

CoreDNS handled service discovery, allowing the frontend to communicate with the backend through its Kubernetes service name.

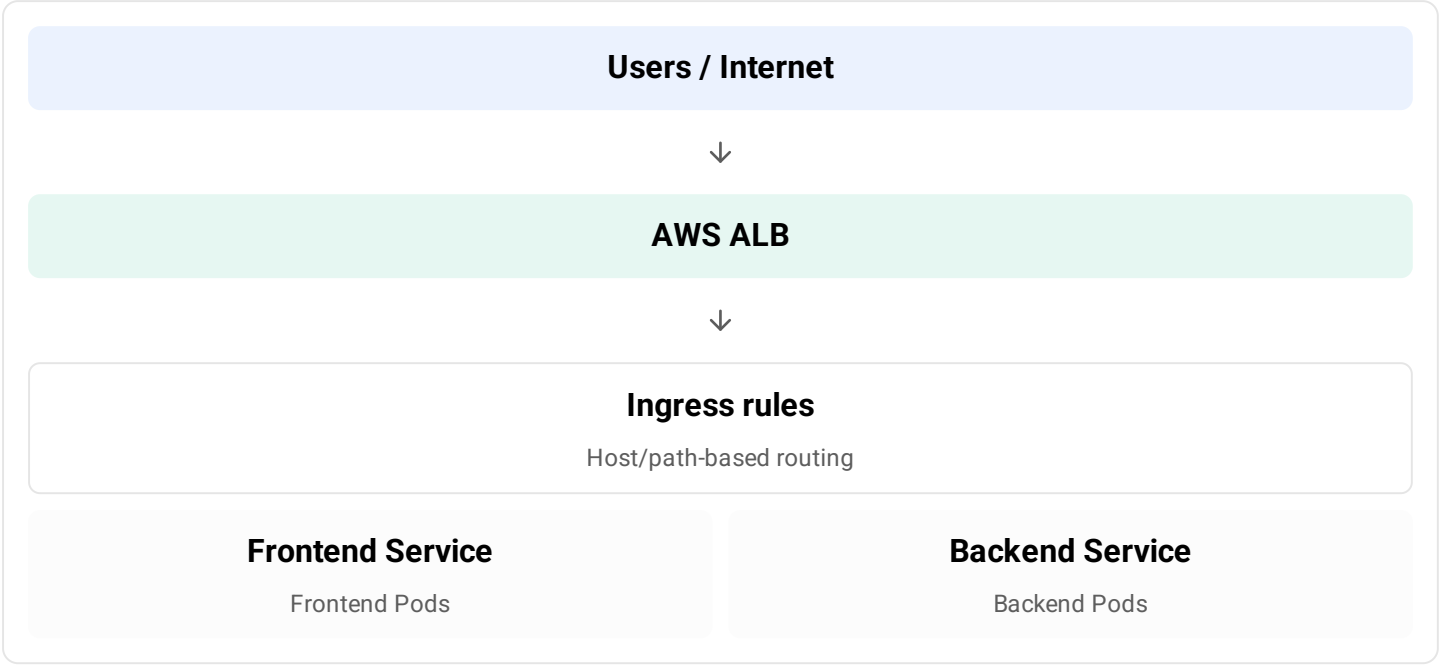
For external access, we used Ingress routing with the appropriate controller. In our AWS environment, AWS Load Balancer Controller could integrate with ALB to distribute incoming traffic to the application.

We also investigated DNS, service endpoint and routing issues whenever application components couldn't communicate."

### 3. Real-world task with an incident

**Task: Expose an application through AWS ALB and route traffic to frontend/backend services.**

Example architecture:




Example Ingress:

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
spec:
  ingressClassName: alb
```

```
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-service
                port:
                  number: 80
```

**Incident: Users receive HTTP 502 or 503 errors.**

Possible reasons:

- No healthy backend pods.
- Incorrect Service selectors.
- Incorrect target port.
- Ingress controller configuration issue.
- ALB target health check failure.
- Security group or network connectivity issue.

Troubleshooting:

```bash
kubectl get svc -A

kubectl get endpoints -A

kubectl get ingress -A

kubectl describe ingress app-ingress

kubectl get pods -n kube-system

kubectl logs -n kube-system \
  deployment/aws-load-balancer-controller
```

For 502/503 errors, inspect the load balancer target health and backend application health as well.

### 4. Common errors and issues

| Issue | Cause | Troubleshooting |
| --- | --- | --- |

| | | |
|---|---|---|
| DNS resolution failure | CoreDNS issue | Check CoreDNS logs |
| Service unavailable | No matching endpoints | Check selectors |
| Connection refused | Wrong target port | Verify ports |
| 502 Bad Gateway | Backend connectivity issue | Check ALB targets |
| 503 Service Unavailable | No healthy endpoints | Inspect readiness |
| Ingress not created | Controller/IAM issue | Check controller logs |
| External traffic blocked | Security groups/NACLs | Verify AWS networking |

**Memory connection:** External users → ALB → Ingress → Service → Pods; internal traffic → Service DNS → CoreDNS.

---

## Responsibility 8: Red Hat OpenShift

Core concept: Enterprise Kubernetes platform

**Resume statement:**

Possess a solid understanding of Red Hat OpenShift, including container orchestration, cluster management, and deploying scalable applications.

### 1. Simple technical definition

Red Hat OpenShift is an enterprise Kubernetes platform developed by Red Hat.

It builds on Kubernetes and provides additional enterprise capabilities such as:

- Web console and developer experience.
- Integrated build and deployment capabilities.
- Security and access control.
- Routes for exposing applications.
- Operator-based cluster management.
- Monitoring and logging integrations.

**Simple distinction:**

Kubernetes provides container orchestration.

OpenShift provides Kubernetes with additional enterprise platform capabilities, tools and policies.

Important OpenShift concepts:

| Concept | Explanation |
|---|---|

| | |
|---|---|
| Project | OpenShift abstraction over Kubernetes namespaces |
| Route | Exposes an application externally |
| SCC | Security Context Constraints |
| Operator | Automates installation and management of applications/platform components |
| BuildConfig | Defines an OpenShift build process |
| Deployment | Manages application deployment |

**2. Storytelling – Project explanation**

"In addition to standard Kubernetes concepts, I understand the OpenShift platform and its enterprise deployment model.

OpenShift provides a developer-friendly interface and additional capabilities for managing containerized applications.

The platform uses projects for workload isolation, Routes for exposing applications externally and security policies to control container execution.

For application deployment, we can use standard Kubernetes workload resources along with OpenShift-specific capabilities where required.

We also need to understand the additional security restrictions and permissions imposed by the platform."

**3. Real-world task with an incident**

**Task: Deploy a containerized application and expose it using an OpenShift Route.**

Workflow:

1. Log in to OpenShift.
2. Select the required project.
3. Deploy the application.
4. Create or verify a Service.
5. Expose the Service through a Route.
6. Verify application accessibility.

Commands:

```
</> Bash

oc login <cluster-url>
```

```
oc project my-project

oc get pods

oc get services

oc expose service frontend-service

oc get routes
```

**Incident: Application container fails because OpenShift restricts its execution permissions.**

For example, an application image expects to run as root or bind to a privileged port.

OpenShift commonly uses restrictive SCC policies and may assign a namespace-specific UID.

Troubleshooting:

```
</> Bash
oc describe pod <pod-name>

oc get scc

oc get events

oc logs <pod-name>
```

Possible solutions:

- Use a non-root compatible container image.
- Make required directories writable by the assigned runtime user/group.
- Avoid hardcoded privileged ports.
- Request appropriate permissions through the platform administrator if genuinely required.

### 4. Common errors and issues

| Issue | Cause | Troubleshooting |
|---|---|---|
| Pod fails to start | SCC restriction | Inspect pod events |
| Image pull failure | Registry authentication | Check image pull Secret |
| Route unavailable | Service/endpoint issue | Check Route and Service |
| Permission denied | Container expects root | Adapt image permissions |
| Build failure | BuildConfig/source issue | Inspect build logs |

| | | |
|---|---|---|
| Operator unhealthy | Reconciliation/configuration issue | Check Operator status |

**Memory connection:** Kubernetes foundation → OpenShift security + Routes + Projects + Operators + Developer platform.

---

# PART 3 – AWS EKS

## Responsibility 9: AWS EKS cluster management

Core concept: Managed Kubernetes on AWS

**Resume statement:**

Good working knowledge of AWS EKS cluster management, including managed node groups, IAM integration, cluster access management, AWS Load Balancer Controller and EBS CSI driver integration.

### 1. Simple technical definition

Amazon EKS is a managed Kubernetes service that allows us to run Kubernetes clusters on AWS without manually managing the Kubernetes control plane.

AWS manages the EKS control plane, while we choose how to provide worker compute capacity.

Important components:

| Component | Purpose |
|---|---|
| EKS Control Plane | Managed Kubernetes API and control plane |
| Managed Node Groups | AWS-managed lifecycle of EC2 worker nodes |
| IAM | AWS identity and permissions |
| EKS Access Entries | Configure IAM principal access to EKS |
| RBAC | Kubernetes authorization |
| AWS Load Balancer Controller | Creates and manages ALB/NLB resources |
| EBS CSI Driver | Provides persistent EBS volumes to pods |

### 2. Storytelling – Project explanation

"Our application was deployed on Amazon EKS to take advantage of managed Kubernetes infrastructure and AWS integration.

We used managed node groups to provide worker node capacity. These nodes hosted our application pods and supported different workloads.

For access management, we integrated IAM identities with EKS access configuration and Kubernetes RBAC to control what users and automation systems could perform.

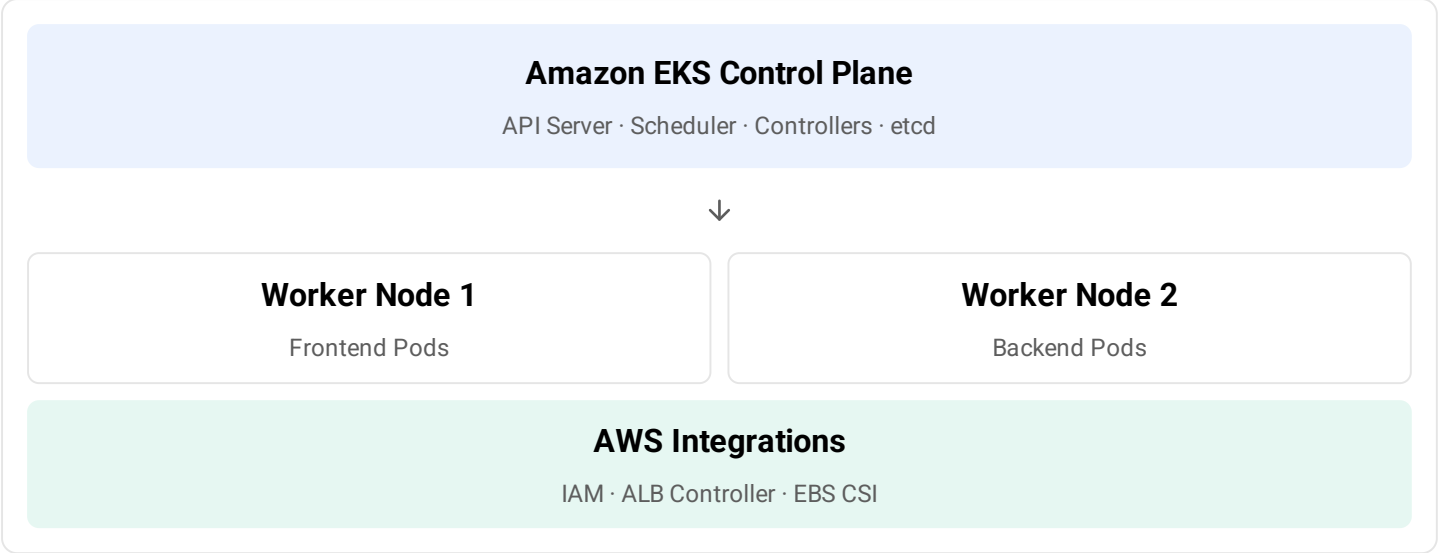
We also used AWS Load Balancer Controller to integrate Kubernetes Ingress resources with AWS load balancers, and EBS CSI Driver for persistent storage requirements.

Our responsibilities included checking node health, verifying cluster access, monitoring workloads and troubleshooting AWS-Kubernetes integration issues."

**3. Real-world task with an incident**

**Task: Deploy an application on an EKS cluster using managed node groups.**

Architecture:




Example commands:

```
</> Bash

aws eks update-kubeconfig \
  --region us-east-1 \
  --name production-cluster

kubectl get nodes

kubectl get pods -A

kubectl get deployments -A
```

Managed node group inspection:

```
</> Bash
```

```
aws eks list-nodegroups \
  --cluster-name production-cluster
```

**Incident: EKS cluster is running, but a developer cannot access it.**

Possible reasons:

- IAM identity not authorized.
- Incorrect EKS access entry.
- Missing Kubernetes RBAC permissions.
- Incorrect kubeconfig.
- Wrong AWS account or region.

Troubleshooting:

```
</> Bash

aws sts get-caller-identity

kubectl config current-context

kubectl get nodes

kubectl auth can-i get pods
```

Check EKS access entries, associated access policies and Kubernetes RBAC bindings.

### 4. Common errors and issues

| Issue | Cause | Troubleshooting |
|---|---|---|
| kubectl unauthorized | IAM access issue | Check access entries |
| Node NotReady | Kubelet or networking issue | Inspect node |
| Node join failure | IAM/bootstrap/network issue | Check node group |
| ALB not created | Controller/IAM issue | Inspect controller logs |
| PVC Pending | EBS CSI issue | Check CSI driver |
| Pods Pending | Insufficient capacity | Check node resources |
| ImagePullBackOff | ECR access issue | Check node/pod permissions |

**Important interview distinction:**

- IAM handles AWS identity and permissions.

- EKS access entries or the configured authentication mechanism establish cluster access.
- Kubernetes RBAC controls what an authenticated identity can do inside Kubernetes.

For EBS CSI, remember that the driver typically requires AWS permissions through an appropriate IAM integration, such as EKS Pod Identity or IRSA.

**Memory connection:** EKS Control Plane → Managed Node Groups → IAM/RBAC → Load Balancer Controller → EBS CSI.

---

# PART 4 – KUBERNETES SCALING AND SCHEDULING

## Responsibility 10: Kubernetes scalability and workload scheduling

Core concept: Performance + Resource optimization

**Resume statement:**

Strong understanding of Kubernetes scalability and workload scheduling, implementing HPA, resource requests and limits, node affinity, taints, tolerations and rolling deployment strategies to improve resource utilization and application availability.

### 1. Simple technical definition

Kubernetes scheduling determines where pods should run, while scalability determines how workloads adjust to changing application demand.

Understand each concept:

| Concept | Purpose |
|---|---|
| HPA | Automatically increases/decreases pod replicas |
| Resource Requests | Resources required for scheduling |
| Resource Limits | Maximum CPU/memory consumption |
| Node Affinity | Attracts or requires pods to run on selected nodes |
| Taints | Restrict which pods can run on a node |
| Tolerations | Allow pods to tolerate matching taints |
| Rolling Updates | Gradually replace old pods with new pods |

### 2. Storytelling – Project explanation

"Our application experienced different traffic patterns throughout the day, so we needed to manage application scaling and workload distribution efficiently.

We configured resource requests and limits to ensure pods had appropriate CPU and memory allocations.

For application scalability, we used HPA to automatically adjust the number of replicas according to CPU and memory utilization.

We also used node affinity and taints/tolerations when certain workloads needed to run on specific node groups or when we wanted to reserve particular nodes for dedicated workloads.

For application releases, we used rolling update strategies to gradually replace existing pods while maintaining availability."

**3. Real-world task with an incident**

**Task: Distribute application workloads and scale during traffic spikes.**

Imagine our cluster has:

- General worker nodes for application workloads.
- Dedicated worker nodes for memory-intensive services.
- Three minimum backend replicas.
- Ten maximum backend replicas.

**A. Node affinity**

Suppose backend workloads should prefer nodes labelled `workload=backend`.

```yaml
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions:
            - key: workload
              operator: In
              values:
                - backend
```

This is a hard scheduling requirement. If no matching eligible node has capacity, the pod remains Pending.

**B. Taints and tolerations**

Taint a node:

```bash
kubectl taint nodes worker-1 \
  dedicated=backend:NoSchedule
```

Pod toleration:

```yaml
tolerations:
  - key: dedicated
    operator: Equal
    value: backend
    effect: NoSchedule
```

A toleration allows the pod to be scheduled onto the tainted node; it does not force it to run there.

**C. HPA**

Configure HPA to increase replicas when CPU utilization increases.

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: backend-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

**Incident: HPA increases replicas, but new pods remain Pending.**

Why?

HPA increases the number of pods, but the cluster might not have enough worker node resources.

Troubleshooting:

```bash
kubectl get hpa

kubectl get pods

kubectl describe pod <pod-name>

kubectl describe node <node-name>

kubectl top nodes
```

We would investigate:

- Available CPU and memory.
- Pod requests.
- Node affinity restrictions.
- Untolerated node taints.
- Node group maximum capacity.
- Cluster Autoscaler or Karpenter status, if configured.

### 4. Common errors and issues

| Issue | Cause | Troubleshooting |
|---|---|---|
| Pods Pending | Insufficient resources | Check requests and node capacity |
| HPA not scaling | Metrics unavailable | Check Metrics Server |
| Node affinity mismatch | No matching labels | Verify node labels |
| Untolerated taint | Missing toleration | Add appropriate toleration |
| CPU throttling | CPU limit too restrictive | Review CPU limit |
| OOMKilled | Memory limit exceeded | Analyze memory |
| Frequent scaling | Fluctuating metrics | Tune HPA behavior |
| Rolling update stuck | New pods not ready | Check readiness probes |

**Important memory distinction:**

- NodeSelector = Simple label-based node selection.
- Node affinity = Advanced required/preferred node selection.
- Taint = Repels pods without toleration.
- Toleration = Allows a pod to tolerate the taint.
- HPA = Scales pods.
- Cluster Autoscaler/Karpenter = Can increase node capacity.

**Memory connection:** Requests → Scheduling → Affinity/Taints → HPA → More Pods → More Node Capacity → Rolling Updates.

---

## Responsibility 11: Infrastructure and container-level troubleshooting

Core concept: Production troubleshooting

**Resume statement:**

Experienced in troubleshooting infrastructure and container-level issues, including SSH connectivity, Ansible execution failures, CrashLoopBackOff, ImagePullBackOff, pod scheduling failures, networking issues and resource exhaustion.

### 1. Simple technical definition

Troubleshooting means identifying the root cause of infrastructure or application failures and restoring normal operation.

Your resume mentions two major areas:

**Infrastructure-level troubleshooting:**

- SSH connection failures.
- Linux service failures.
- Ansible execution failures.
- AWS network and access issues.
- Disk and memory utilization.

**Container-level troubleshooting:**

- CrashLoopBackOff.
- ImagePullBackOff.
- Pending pods.
- DNS and Service failures.
- CPU/memory exhaustion.

### 2. Storytelling – Project explanation

"In our application environment, infrastructure and application issues could affect deployment and service availability.

We followed a systematic troubleshooting approach rather than immediately restarting services or pods.

For infrastructure issues, we checked SSH connectivity, server resource utilization, system services and Ansible execution logs.

For Kubernetes issues, we first checked pod status, events and container logs. Depending on the failure, we investigated image availability, application startup errors, resource constraints, scheduling rules and network connectivity.

After identifying the root cause, we applied the required correction and validated that the application returned to a healthy state."

**3. Real-world task with an incident**

**Task: Troubleshoot a backend application that is unavailable after deployment.**

Imagine users report that the backend API is unavailable after a release.

Start with:

```bash
kubectl get pods -n production

kubectl get deployments -n production

kubectl get services -n production

kubectl get events -n production
```

Now identify the problem based on the pod status.

**Scenario A: CrashLoopBackOff**

Meaning: Container repeatedly starts and crashes.

Possible reasons:

- Application startup exception.
- Incorrect environment variables.
- Missing dependencies.
- Incorrect startup command.

- Database connectivity failure.

Commands:

```bash
kubectl logs <pod-name>

kubectl logs <pod-name> --previous

kubectl describe pod <pod-name>
```

**Scenario B: ImagePullBackOff**

Meaning: Kubernetes cannot successfully pull the required container image.

Possible reasons:

- Incorrect image name.
- Incorrect tag.
- ECR authentication issue.
- Network connectivity problem.
- Missing registry permissions.

Commands:

```bash
kubectl describe pod <pod-name>

kubectl get events -n production
```

Check the image reference, ECR repository and applicable IAM permissions.

**Scenario C: Pending pod**

Meaning: Kubernetes cannot find an eligible node on which to schedule the pod.

Possible reasons:

- Insufficient CPU or memory.
- Node taints.
- Node affinity mismatch.
- Unavailable persistent volume.
- Node group capacity issue.

Commands:

```bash
kubectl describe pod <pod-name>

kubectl get nodes

kubectl describe nodes

kubectl top nodes
```

**Scenario D: SSH connectivity failure**

Suppose Ansible cannot connect to an EC2 server.

```bash
ssh -i key.pem ec2-user@<public-ip>

ansible all -m ping -i inventory
```

Check:

- SSH key and username.
- Security group inbound rules.
- Network ACLs and routing.
- Instance status.
- SSH daemon.
- Correct private/public IP.

**4. Common errors and issues**

| Error | Meaning | First troubleshooting step |
|---|---|---|
| Connection timed out | Network connectivity issue | Check security group/routing |
| Permission denied | Authentication/authorization issue | Check SSH key/user |
| CrashLoopBackOff | Container repeatedly crashes | Previous logs |
| ImagePullBackOff | Image pull unsuccessful | Pod events |
| Pending | Pod unschedulable or blocked | Describe pod |
| OOMKilled | Memory limit exceeded | Check memory metrics |

| | | |
|---|---|---|
| DNS failure | Name resolution issue | Check CoreDNS |
| Service unavailable | No healthy endpoints | Check Service selectors |
| DiskPressure | Node disk pressure | Check disk usage |
| Node NotReady | Node health problem | Check kubelet/runtime |

**Memory connection:** Identify symptom → Check status → Inspect events → Read logs → Find root cause → Apply fix → Validate.

---

## Responsibility 12: End-to-end DevOps toolchain integration

Core concept: Complete application delivery

**Resume statement:**

Good understanding of integrating Ansible, Kubernetes, Jenkins, Docker, Git and AWS into DevOps workflows to automate infrastructure configuration, container deployments and continuous application delivery.

### 1. Simple technical definition

DevOps toolchain integration means connecting multiple tools to automate the complete application delivery lifecycle.

Each tool has a particular responsibility:

| Tool | Responsibility |
|---|---|
| Git | Source code version control |
| Jenkins | CI/CD pipeline automation |
| Docker | Containerization |
| Ansible | Server configuration automation |
| AWS | Cloud infrastructure |
| ECR | Container image registry |
| Kubernetes/EKS | Container orchestration |
| Helm | Application packaging and deployment |

The important thing is understanding how data and artifacts move between these tools.

### 2. Storytelling – Project explanation

"Our DevOps workflow involved multiple tools working together to automate application delivery.

Developers maintained the application source code in Git. Whenever code changes were pushed, Jenkins triggered the pipeline and performed the required build and validation steps.

After successful compilation, Docker was used to package the application into a container image, which was pushed to Amazon ECR.

For infrastructure configuration and server administration, we used Ansible automation. For containerized application deployment, we used Kubernetes and Helm on AWS EKS.

The pipeline also included deployment validation to ensure that the new application version was running successfully.

This integrated approach helped standardize deployments and reduce manual operational activities."

**3. Real-world task with an incident**

**Task: Deploy a new frontend application version into production EKS.**

### Complete DevOps delivery pipeline

1. **Git**Developer pushes application code.
2. **Jenkins**Checkout, build, test and validate.
3. **Docker**Build and tag the application image.
4. **Amazon ECR**Store the container image.
5. **Helm / Kubernetes**Deploy the image to EKS.
6. **AWS / Ansible**Manage supporting infrastructure and server configuration.
7. **Validation**Check pod health, service endpoints and application availability.

Example simplified Jenkins pipeline:

```groovy
</> Groovy

pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                git 'https://github.com/org/frontend.git'
            }
        }

        stage('Build') {
            steps {
                sh 'npm ci'
```

```
                sh 'npm run build'
            }
        }

        stage('Docker Build') {
            steps {
                sh 'docker build -t frontend:${BUILD_NUMBER} .'
            }
        }

        stage('Push to ECR') {
            steps {
                echo 'Authenticate and push image to ECR'
            }
        }

        stage('Deploy to EKS') {
            steps {
                echo 'Deploy application using Helm'
            }
        }

        stage('Validation') {
            steps {
                sh 'kubectl rollout status deployment/frontend'
            }
        }
    }
}
```

For a production implementation, use immutable image tags, securely managed AWS credentials, controlled deployment permissions and explicit release approvals where required.

**Incident: Jenkins reports successful deployment, but users cannot access the application.**

This is a very realistic distributed troubleshooting scenario.

We should not assume that a successful Jenkins pipeline means the application is fully functional.

Investigate layer by layer:

1. **Jenkins:** Did the deployment command actually succeed?
2. **ECR:** Is the correct image available?
3. **Kubernetes:** Are pods running and ready?
4. **Service:** Are endpoints available?
5. **Ingress/ALB:** Are targets healthy?
6. **Application:** Is the backend responding correctly?

Commands:

```bash
kubectl get pods -n production

kubectl get deployments -n production

kubectl get svc -n production

kubectl get endpoints -n production

kubectl get ingress -n production

kubectl rollout status deployment/frontend
```

Then inspect the application logs and ALB target health.

#### 4. Common errors and issues

| Issue | Cause | Resolution |
|---|---|---|
| Git checkout failure | Credentials or branch problem | Verify SCM |
| Jenkins build failure | Dependency/compiler issue | Inspect build logs |
| Docker build failure | Dockerfile/context issue | Validate Docker build |
| ECR push failure | IAM authentication/permission issue | Check ECR access |
| ImagePullBackOff | Image or registry issue | Inspect pod events |
| Helm upgrade failure | Invalid chart/configuration | Run lint/template |
| Pods Pending | Insufficient cluster capacity | Check scheduling |
| Ingress failure | ALB/controller issue | Check target health |
| Application unavailable | Application/dependency issue | Inspect logs and health |

**Memory connection:** Git → Jenkins → Docker → ECR → Helm → EKS → ALB → Users.

---

## Final revision – Connect all 12 responsibilities

This is the bigger picture you should remember during your client interview.

### Your complete DevOps engineering workflow

1. **Ansible automation**Build reusable roles and playbooks to automate server configurations.
2. **Linux administration**Configure RHEL/Ubuntu servers, packages, users, permissions and services.
3. **AWS automation**Discover EC2 instances, configure servers and manage AWS resources.
4. **Jenkins integration**Trigger Ansible automation and application deployment workflows.
5. **Kubernetes architecture**Understand control plane, worker nodes, pods and workload management.
6. **Kubernetes workloads**Manage Deployments, StatefulSets, ConfigMaps, Secrets and Helm.
7. **Networking**Maintain DNS, Services, Ingress and external traffic routing.
8. **OpenShift**Understand enterprise Kubernetes, Routes, SCCs and Operators.
9. **AWS EKS**Manage managed node groups, IAM, ALB Controller and EBS CSI.
10. **Scaling and scheduling**Configure HPA, resources, affinity, taints and tolerations.
11. **Troubleshooting**Investigate infrastructure, container, networking and resource failures.
12. **End-to-end integration**Connect Git, Jenkins, Docker, ECR, Ansible, Helm and EKS.

## The 15 questions I would prioritize before your client interview

| Priority | Question |
|---|---|
| 1 | Explain Ansible roles and their directory structure. |
| 2 | How do you use handlers, templates and variables? |
| 3 | How do you perform Linux patching using Ansible? |
| 4 | How does Ansible dynamic inventory work with AWS? |
| 5 | How do you integrate Ansible with Jenkins? |
| 6 | Explain Kubernetes architecture. |
| 7 | Deployment vs StatefulSet vs ReplicaSet? |
| 8 | ConfigMap vs Secret? |
| 9 | Explain Kubernetes networking and DNS. |
| 10 | How does AWS EKS authentication work? |
| 11 | Explain AWS Load Balancer Controller and EBS CSI. |
| 12 | HPA vs Cluster Autoscaler vs Karpenter? |
| 13 | Node affinity vs taints and tolerations? |
| 14 | How do you troubleshoot CrashLoopBackOff and ImagePullBackOff? |
| 15 | Explain your end-to-end CI/CD pipeline. |

## My final advice: Prepare for depth, not just breadth

For your experience profile, I would particularly focus on these five areas:

1. **Ansible:** Roles, variables, handlers, Linux administration, dynamic inventory and patching.
2. **Kubernetes:** Architecture, deployments, scheduling, networking and troubleshooting.
3. **AWS EKS:** IAM, managed node groups, ALB Controller and EBS CSI.
4. **Jenkins:** Pipeline stages, Docker builds, ECR integration and deployments.
5. **Production troubleshooting:** How to identify a problem, investigate logs/events, find root causes and validate recovery.

One important thing: don't claim that you independently administered a production EKS cluster or executed production patching if you haven't. You can still demonstrate strong practical understanding by explaining the exact workflow, commands, configurations, risks and troubleshooting decisions.

**The interviewer's real test is whether you understand why something is configured, what happens when it fails, and how you would safely resolve it.**